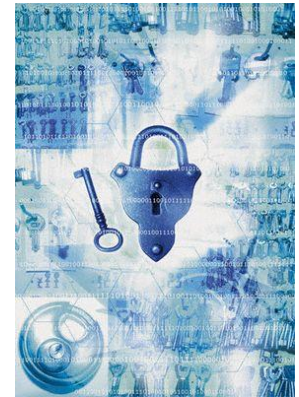


Information Security



Hash functions

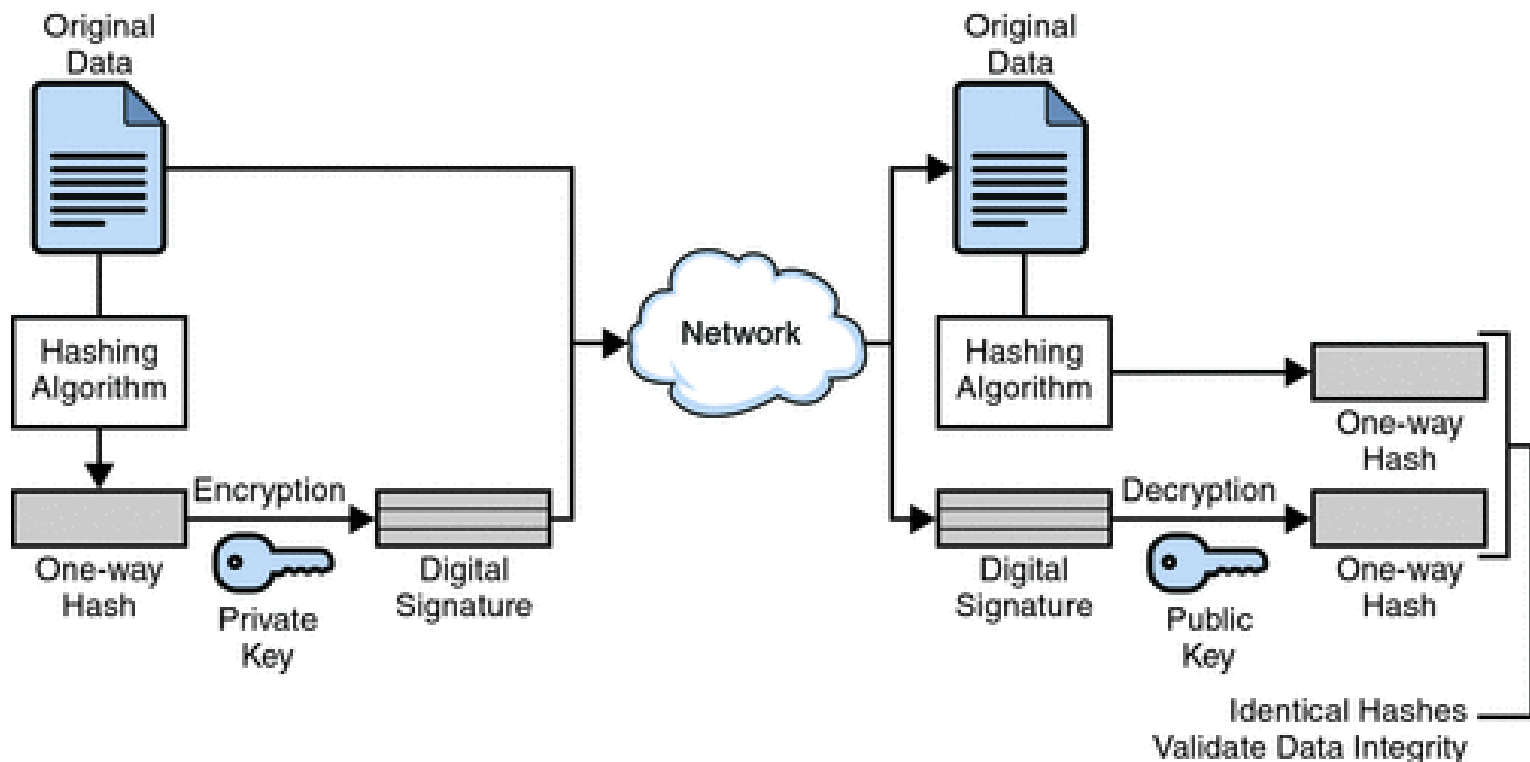
AI컴퓨터공학부
김희열

Hash Function Motivation

- Suppose Alice signs M
 - Alice sends M and $S = [M]_{\text{Alice}}$ to Bob
 - Bob verifies that $M = \{S\}_{\text{Alice}}$
 - Is it OK to just send S ?
- If M is big, $[M]_{\text{Alice}}$ is costly to compute

Hash Function Motivation

- Alice signs $h(M)$, where $h(M)$ is much smaller than M
 - Alice sends M and $S = [h(M)]_{\text{Alice}}$ to Bob
 - Bob verifies that $h(M) = \{S\}_{\text{Alice}}$



Crypto Hash Function

- Crypto hash function $h(x)$ must provide
 - **Compression**
 - output length is small
 - **Efficiency**
 - $h(x)$ is easy to compute for any x
 - **One-way**
 - given a value y it is infeasible to find an x such that $h(x) = y$
 - **Weak collision resistance**
 - given x and $h(x)$, infeasible to find $y \neq x$ such that $h(y) = h(x)$
 - **Strong collision resistance**
 - infeasible to find any x and y , with $x \neq y$ such that $h(x) = h(y)$
 - Lots of collisions exist, but hard to find one

Pre-Birthday Problem

- Suppose N people in a room
- How large must N be?
 - The probability someone has same birthday as me is $\geq 1/2$
 - Solve: $1/2 = 1 - (364/365)^N$ for N
 - $N = 253$

Birthday Problem

- How many people must be in a room?
 - The probability is $\geq 1/2$ that two or more have same birthday
 - $1 - 365/365 \cdot 364/365 \cdot \dots \cdot (365-N+1)/365$
 - Set equal to $1/2$ and solve: **N = 23**
- Surprising? A paradox?
- Maybe not
 - “Should be” about $\sqrt{(365)}$ since we compare all **pairs** x and y

Hashes and Birthday Problem

- If $h(x)$ is N bits, then 2^N different hash values are possible
- $\sqrt{2^N} = 2^{N/2}$
- So, hash about $2^{N/2}$ random values and you expect to find a collision

- Secure N bit symmetric key requires 2^{N-1} work to “break”
- Secure N bit hash requires $2^{N/2}$ work to “break”

Non-crypto Hash (1)

- Data $X = (X_0, X_1, X_2, \dots, X_{n-1})$, each X_i is a byte
- Hash $h(X) = X_0 + X_1 + X_2 + \dots + X_{n-1}$
- Is this secure?
- Example: $X = (10101010, 00001111)$
- Hash $h(X)$ is 10111001
- But so is hash of $Y = (00001111, 10101010)$
- Easy to find collisions, so **not** secure...

Non-crypto Hash (2)

- Data $X = (X_0, X_1, X_2, \dots, X_{n-1})$
- Hash $h(X) = nX_0 + (n-1)X_1 + (n-2)X_2 + \dots + 1 \cdot X_{n-1}$
- Is this hash secure?
- At least
 - $h(10101010, 00001111) \neq h(00001111, 10101010)$
- But hash of $(00000001, 00001111)$ is same as hash of $(00000000, 00010001)$
- Not one-way
- This hash is used in the (non-crypto) application [rsync](#)

Non-crypto Hash (3)

- Cyclic Redundancy Check (CRC)
 - To detect accidental changes of data
 - Used in networks & storage devices
- Essentially, CRC is the remainder in a long division problem
- Good for detecting burst **errors caused by transmission noise**
- But easy to construct collisions
- CRC sometimes mistakenly used in crypto applications (WEP)
- **CRC is not designed to detect intentional tampering**

Popular Crypto Hashes

- **MD5**
 - Invented by Rivest
 - 128 bit output
 - Note: MD5 collision recently found
- **SHA-1**
 - A US government standard (similar to MD5)
 - 160 bit output
- Many others hashes, but MD5 and SHA-1 most widely used
- Hashes work by hashing message in blocks

SHA

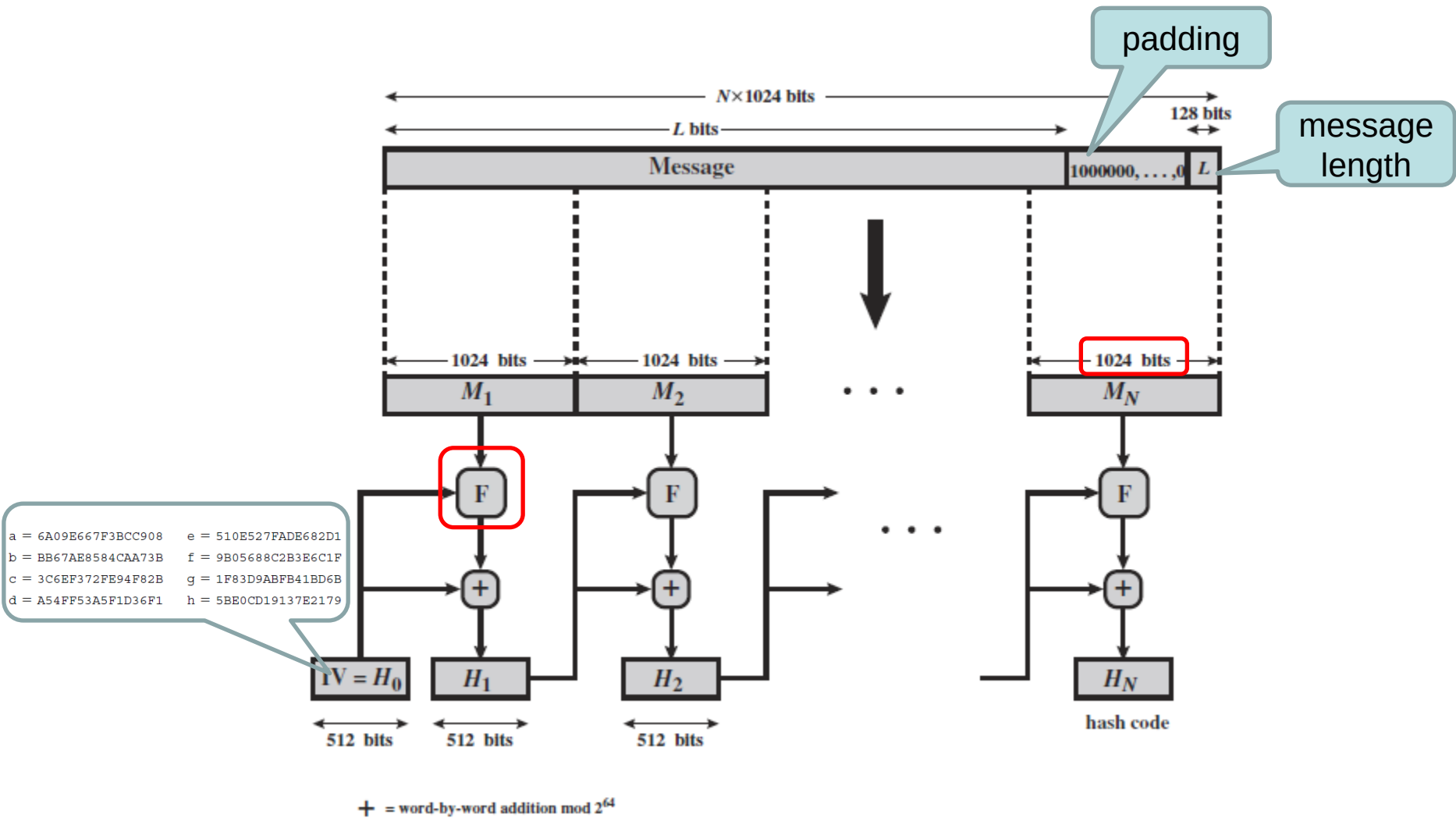
- SHA-0
 - Based on MD4
 - Weakness was discovered
- SHA-1
 - Not yet broken, but considered insecure
- SHA-2
 - Revised version of SHA-1, in 2002

Table 11.3 Comparison of SHA Parameters

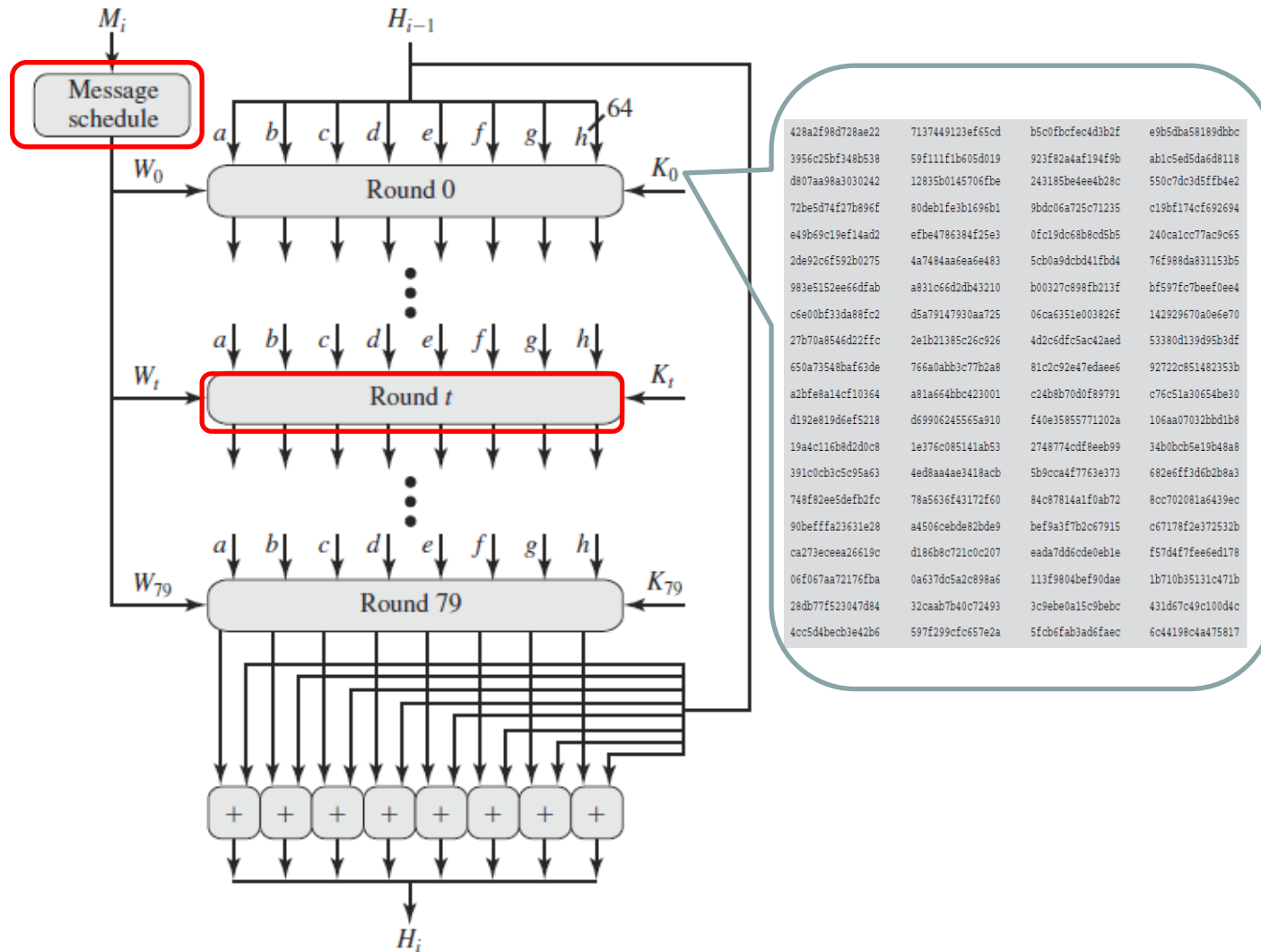
	SHA-1	SHA-224	SHA-256	SHA-384	SHA-512
Message Digest Size	160	224	256	384	512
Message Size	$< 2^{64}$	$< 2^{64}$	$< 2^{64}$	$< 2^{128}$	$< 2^{128}$
Block Size	512	512	512	1024	1024
Word Size	32	32	32	64	64
Number of Steps	80	64	64	80	80

Note: All sizes are measured in bits.

SHA-512



SHA-512



SHA-512

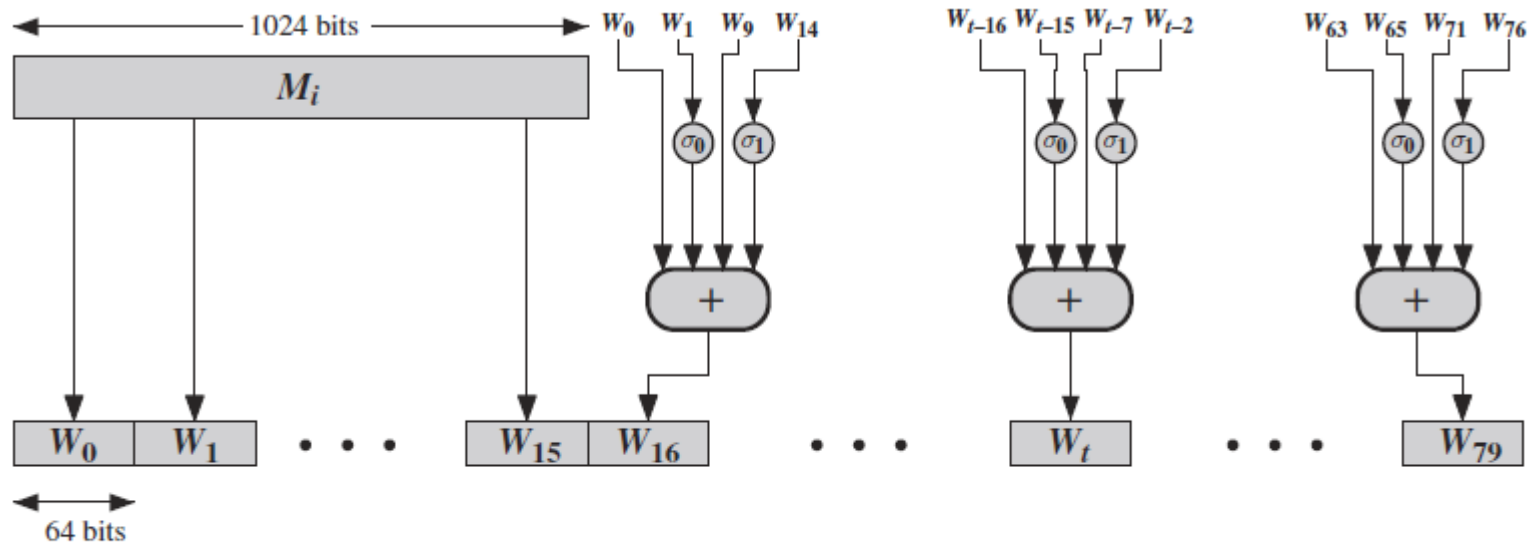


Figure 11.12 Creation of 80-word Input Sequence for SHA-512 Processing of Single Block

SHA-512

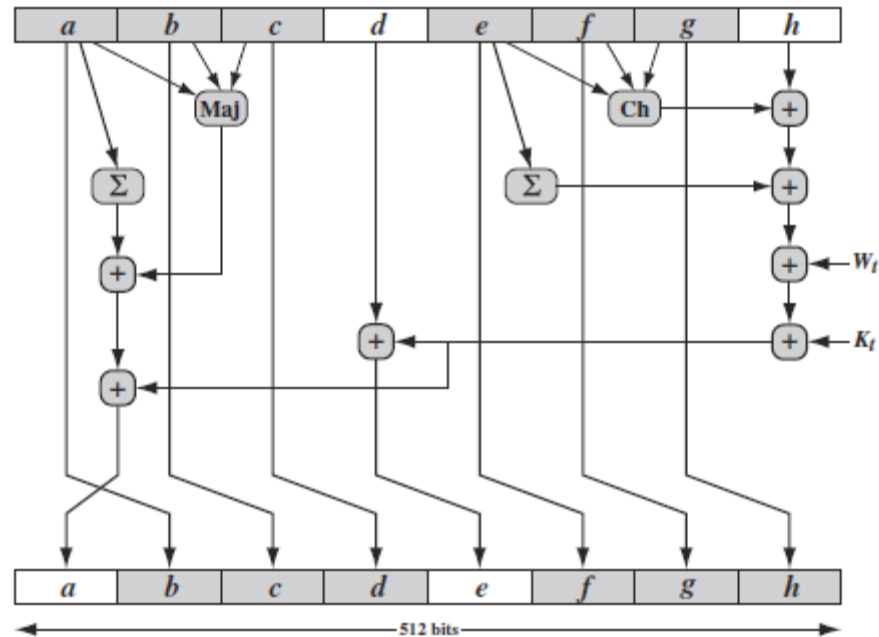


Figure 11.11 Elementary SHA-512 Operation (single round)

SHA-512 example

- Input message
 - abc (in ASCII character)
 - = 01100001 01100010 01100011

- After padding

```
6162638000000000 0000000000000000 0000000000000000 0000000000000000
0000000000000000 0000000000000000 0000000000000000 0000000000000000
0000000000000000 0000000000000000 0000000000000000 0000000000000000
0000000000000000 0000000000000000 0000000000000000 0000000000000018
```

- Initial values (H0)

<i>a</i>	6a09e667f3bcc908
<i>b</i>	bb67ae8584caa73b
<i>c</i>	3c6ef372fe94f82b
<i>d</i>	a54ff53a5f1d36f1
<i>e</i>	510e527fade682d1
<i>f</i>	9b05688c2b3e6c1f
<i>g</i>	1f83d9abfb41bd6b
<i>h</i>	5be0cd19137e2179

SHA-512 example

- After 80 rounds

```
73a54f399fa4b1b2 10d9c4c4295599f6 d67806db8b148677 654ef9abec389ca9
d08446aa79693ed7 9bb4d39778c07f9e 25c96a7768fb2aa3 ceb9fc3691ce8326
```

- Hash value

```
 $H_{1,0} = 6a09e667f3bcc908 + 73a54f399fa4b1b2 = ddaf35a193617aba$ 
 $H_{1,1} = bb67ae8584caa73b + 10d9c4c4295599f6 = cc417349ae204131$ 
 $H_{1,2} = 3c6ef372fe94f82b + d67806db8b148677 = 12e6fa4e89a97ea2$ 
 $H_{1,3} = a54ff53a5f1d36f1 + 654ef9abec389ca9 = 0a9eeee64b55d39a$ 
 $H_{1,4} = 510e527fade682d1 + d08446aa79693ed7 = 2192992a274fc1a8$ 
 $H_{1,5} = 9b05688c2b3e6c1f + 9bb4d39778c07f9e = 36ba3c23a3feebbd$ 
 $H_{1,6} = 1f83d9abfb41bd6b + 25c96a7768fb2aa3 = 454d4423643ce80e$ 
 $H_{1,7} = 5be0cd19137e2179 + ceb9fc3691ce8326 = 2a9ac94fa54ca49f$ 
```

- Resulting 512-bit hash value(or message digest)

```
ddaf35a193617aba cc417349ae204131 12e6fa4e89a97ea2 0a9eeee64b55d39a
2192992a274fc1a8 36ba3c23a3feebbd 454d4423643ce80e 2a9ac94fa54ca49f
```

Crypto Hash Design

- Desired property: **avalanche effect**
 - Change to 1 bit of input should affect about half of output bits
 - Why?
- Crypto hash functions consist of some number of rounds
- Want security and speed
 - Avalanche effect after few rounds
 - But simple rounds
- Analogous to design of block ciphers

Crypto Hash Design

- SHA-512 hash value of input “abc”

```
ddaf35a193617aba cc417349ae204131 12e6fa4e89a97ea2 0a9eeee64b55d39a
2192992a274fc1a8 36ba3c23a3feebbd 454d4423643ce80e 2a9ac94fa54ca49f
```

- SHA-512 hash value of input “cbc”

```
531668966ee79b70 0b8e593261101354 4273f7ef7b31f279 2a7ef68d53f93264
319c165ad96d9187 55e6a204c2607e27 6e05cdf993a64c85 ef9e1e125c0f925f
```

- 254 bits are differ between the two values above
- Good avalanche effect

SHA-3

- SHA-1
 - insecure
- SHA-2
 - Share the same structure and operations as SHA-1
 - No significant attacks on SHA-2 until now
- SHA-3
 - Next generation NIST hash function
 - Announced in 2012
 - Widely used
 - Based on sponge construction

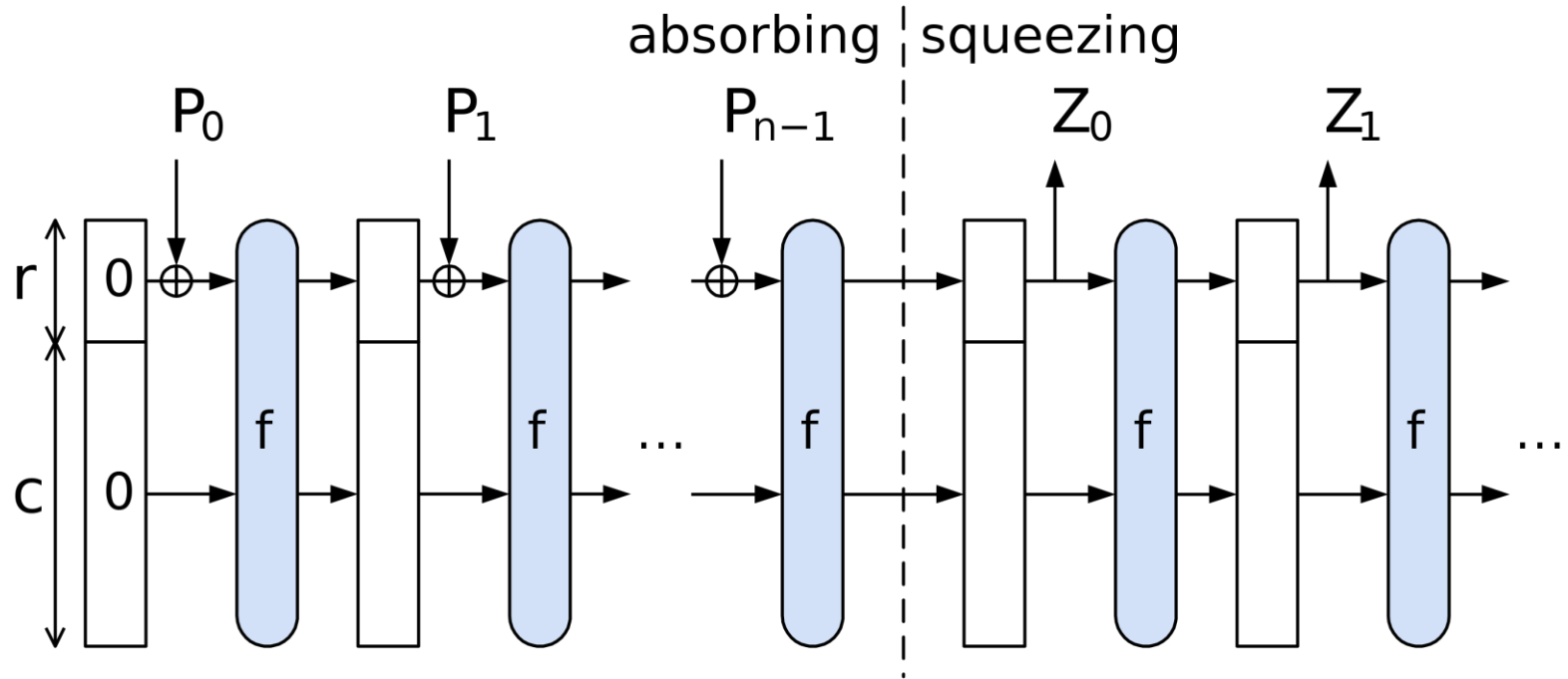
SHA-3

- NIST is having an ongoing competition for SHA-3, the next generation of standard hash algorithms
- 2007: Request for submissions of new hash functions
- 2008: Submissions deadline. Received 64 entries. Announced first-round selections of 51 candidates.
- 2009: After First SHA-3 candidate conference in Feb, announced 14 Second Round Candidates in July.
- 2010: After one year public review of the algorithms, hold second SHA-3 candidate conference in Aug. Announced 5 Third-round candidates in Dec.
- 2011: Public comment for final round
- 2012: October 2, NIST selected SHA3
 - Keccak (pronounced “catch-ack”) created by Guido Bertoni, Joan Daemen and Gilles Van Assche, Michaël Peeters

SHA-3

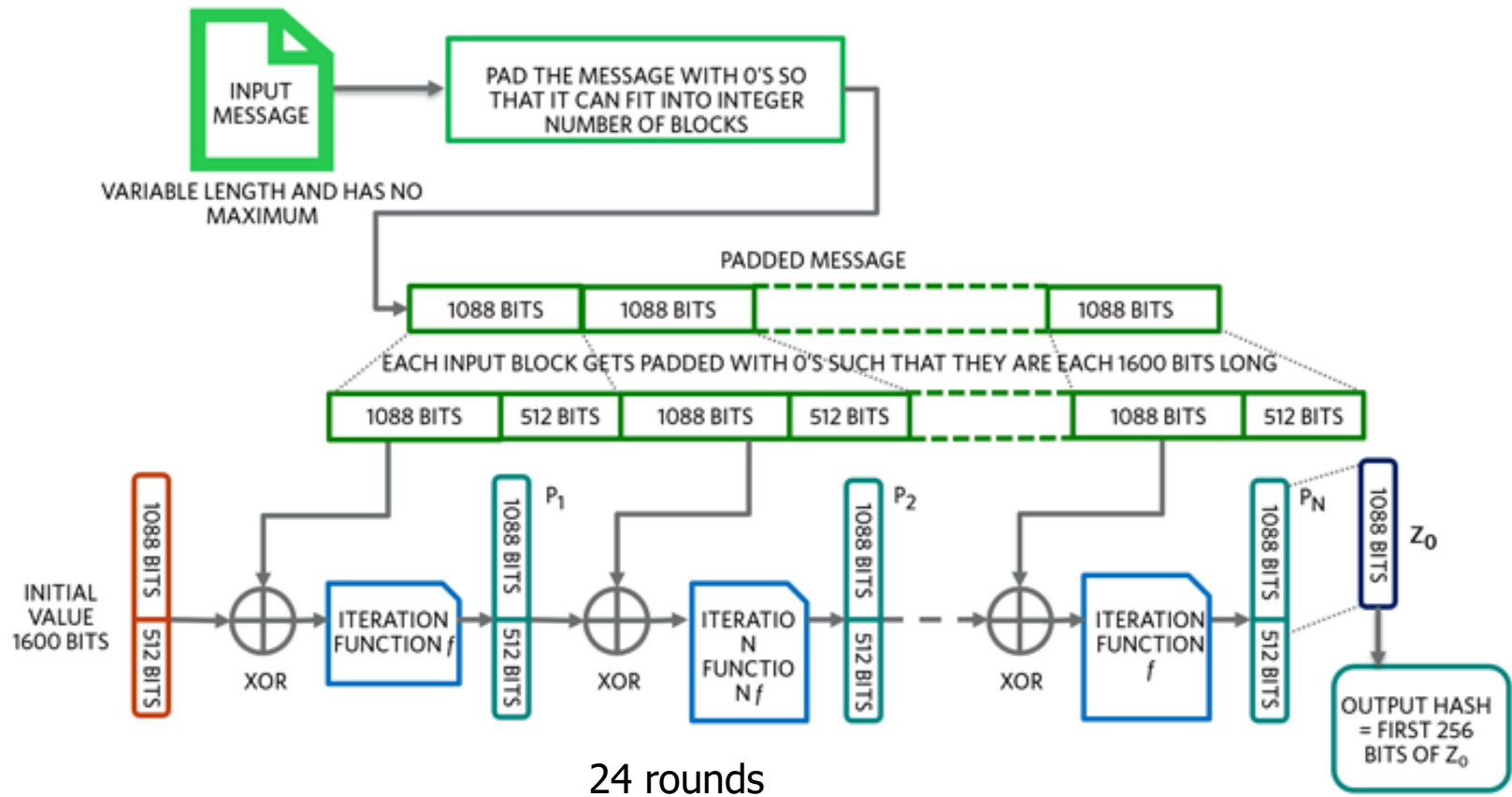
- Sponge construction

- Input bit stream of any length $\rightarrow$ output bit stream of any desired length
- f : block transformation (permutation) using XOR, AND, NOT



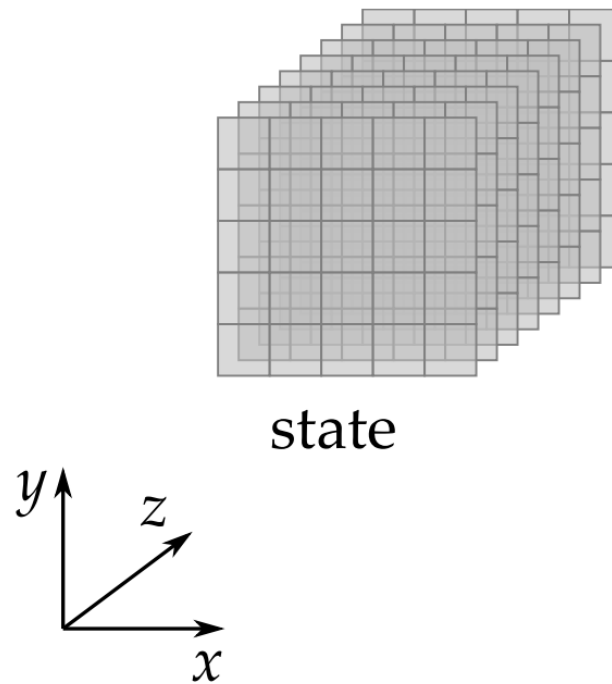
SHA-3

- SHA3-256



SHA-3

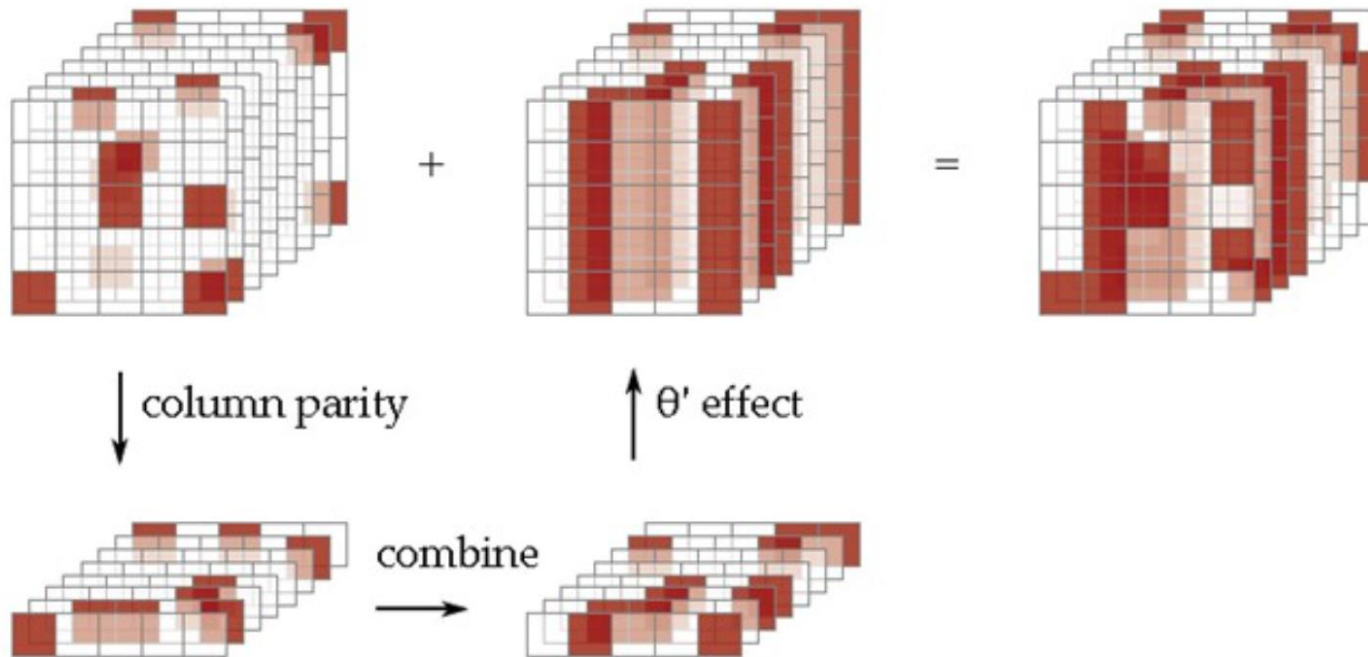
- Keccak-f
 - θ (theta) $\rightarrow \rho$ (rho) $\rightarrow \pi$ (pi) $\rightarrow \chi$ (chi) $\rightarrow \iota$ (iota)
 - State representation



SHA-3

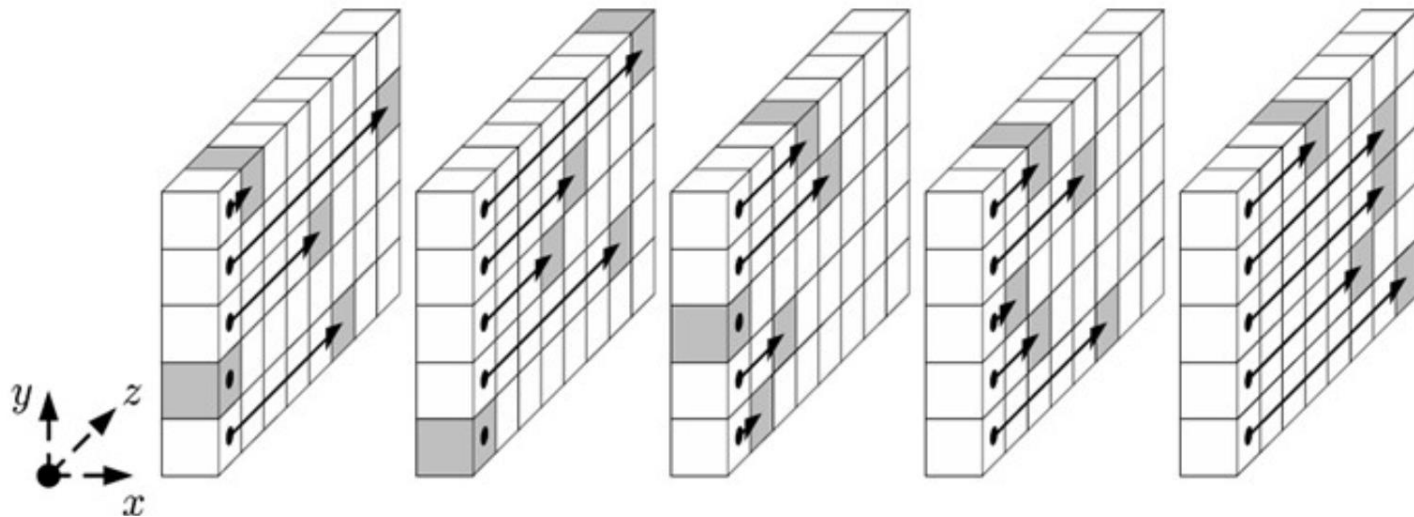
- θ (theta)
 - Diffusion across columns of state

- $$b_{x,y,z} = b_{x,y,z} \oplus c_{x-1,z} \oplus c_{x+1,z}$$



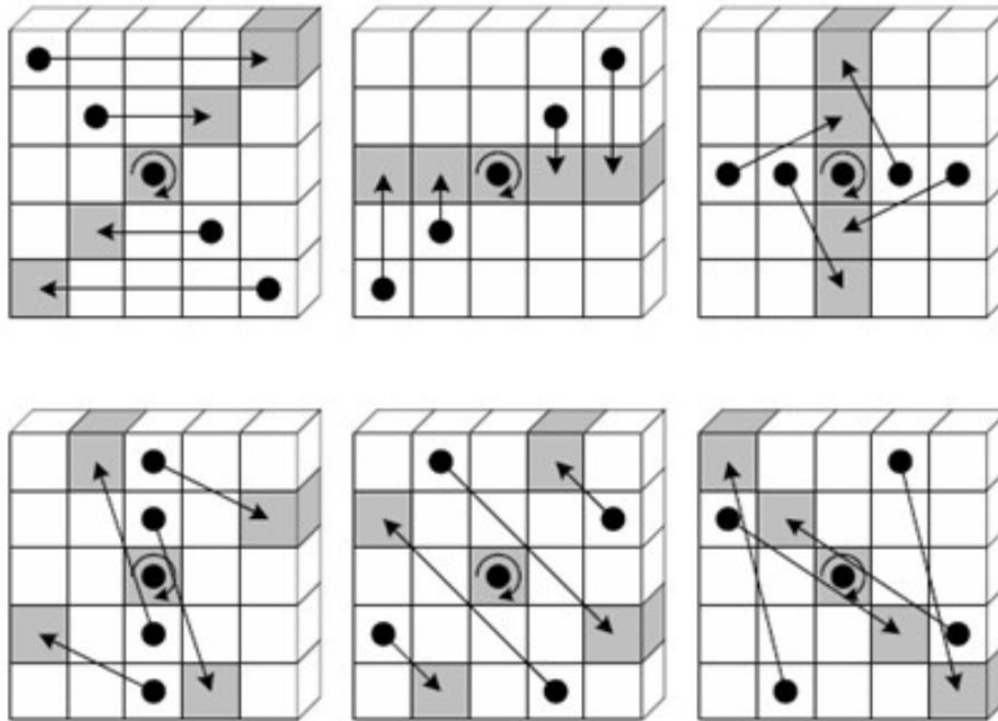
SHA-3

- ρ (rho)
 - Complex diffusion along lanes



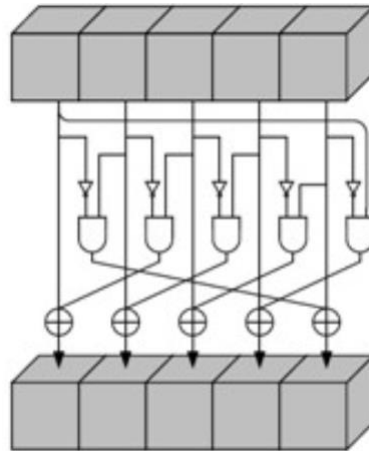
SHA-3

- π (π)
 - Disturb horizontal/vertical alignments in slice



SHA-3

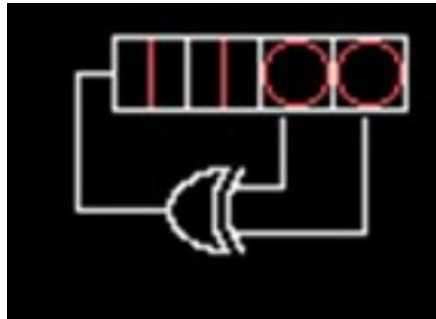
- χ (chi)
 - Mixing bits withing same row



$$\bullet \ b_{x,y,z} = b_{x,y,z} \oplus (\neg b_{(x+1),y,z} \wedge b_{(x+2),y,z})$$

SHA-3

- I (iota)
 - Simple linear shift register



Comparison of SHA Functions

Algorithm and variant		Output size (bits)	Internal state size (bits)	Block size (bits)	Rounds	Operations	Security against collision attacks (bits)	Security against length extension attacks (bits)	Performance on Skylake (median cpb) ^[58]		First published
									Long messages	8 bytes	
MD5 (as reference)		128	128 (4 × 32)	512	64	And, Xor, Rot, Add (mod 2 ³²), Or	≤ 18 (collisions found) ^[59]	0	4.99	55.00	1992
SHA-0		160	160 (5 × 32)	512	80	And, Xor, Rot, Add (mod 2 ³²), Or	< 34 (collisions found)	0	≈ SHA-1	≈ SHA-1	1993
SHA-1							< 63 (collisions found) ^[60]		3.47	52.00	1995
SHA-2	SHA-224	224	256 (8 × 32)	512	64	And, Xor, Rot, Add (mod 2 ³²), Or, Shr	112	32	7.62	84.50	2004
	SHA-256	256					128	0	7.63	85.25	2001
	SHA-384	384	512 (8 × 64)	1024	80	And, Xor, Rot, Add (mod 2 ⁶⁴), Or, Shr	192	128 (≤ 384)	5.12	135.75	2001
	SHA-512	512	512 (8 × 64)	1024	80	And, Xor, Rot, Add (mod 2 ⁶⁴), Or, Shr	256	0 ^[61]	5.06	135.50	2001
	SHA-512/224 SHA-512/256	224 256	512 (8 × 64)	1024	80	And, Xor, Rot, Add (mod 2 ⁶⁴), Or, Shr	112 128	288 256	≈ SHA-384	≈ SHA-384	2012
SHA-3	SHA3-224	224	1600 (5 × 5 × 64)	1152	24 ^[62]	And, Xor, Rot, Not	112	448	8.12	154.25	2015
	SHA3-256	256		1088			128	512	8.59	155.50	
	SHA3-384	384		832			192	768	11.06	164.00	
	SHA3-512	512		576			256	1024	15.88	164.00	
	SHAKE128	d (arbitrary)		1344			min(d/2, 128)	256	7.08	155.25	
	SHAKE256	d (arbitrary)		1088			min(d/2, 256)	512	8.59	155.50	

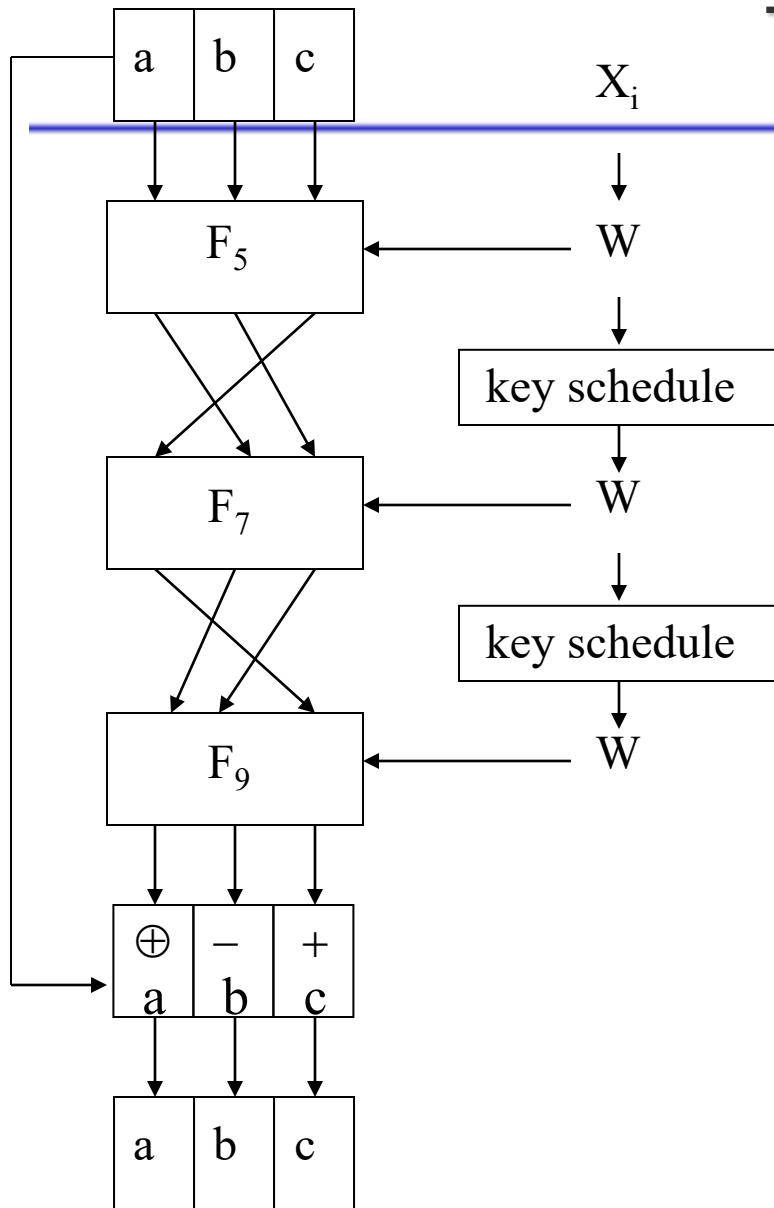
Tiger Hash

- “Fast and strong”
- Designed by
 - Ross Anderson and Eli Biham
- Design criteria
 - Secure
 - Optimized for **64-bit** processors
 - Easy replacement for MD5 or SHA-1

Tiger Hash

- Like MD5/SHA-1, input divided into 512 bit blocks (padded)
- Unlike MD5/SHA-1, output is **192 bits** (three 64-bit words)
- Intermediate rounds are all 192 bits
- 4 S-boxes, each maps 8 bits to 64 bits
- A “key schedule” is used

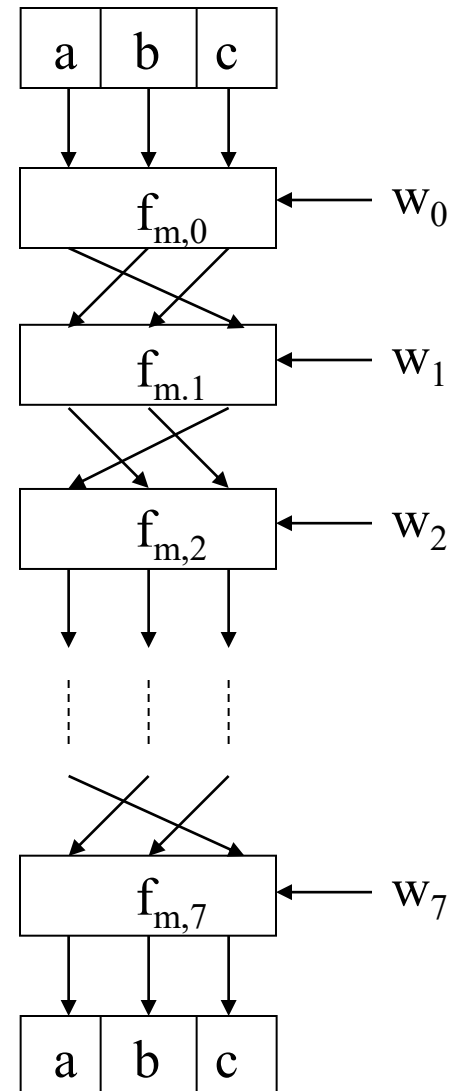
Tiger Outer Round



- Input is X
 - $X = (X_0, X_1, \dots, X_{n-1})$
 - X is padded
 - Each X_i is 512 bits
- n iterations of diagram
 - One for each input block
- Initial (a, b, c) constants
 - $a = 0x123456789ABCDEF$
 - $b = 0xFEDCBA987654321$
 - $c = 0xE096A5B4C3B2E187$
- Final (a, b, c) is hash
- Looks like block cipher!

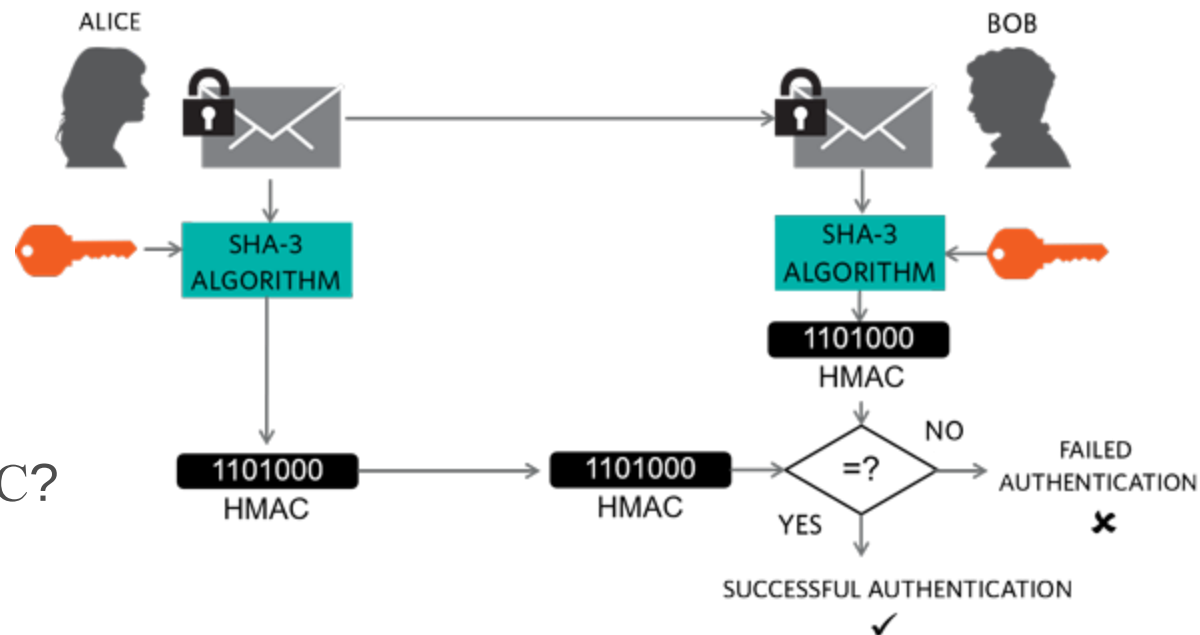
Tiger Inner Rounds

- Each F_m consists of precisely **8 rounds**
- 512 bit input W to F_m
 - $W=(w_0, w_1, \dots, w_7)$
 - W is one of the input blocks X_i
- All lines are 64 bits
- The $f_{m,i}$ depend on the S-boxes



HMAC

- Can compute a MAC value using a “hashed MAC” or **HMAC**
 - What is MAC?
- HMAC is an example of a **keyed** hash
 - Why do we need a key?



- How to compute HMAC?
 - Two obvious choices
 - $h(K, M)$
 - $h(M, K)$

HMAC

- Should we compute HMAC as $h(K,M)$?
- Hashes computed in blocks
 - $h(B_1, B_2) = F(F(A, B_1), B_2)$ for some F and constant A
 - Then $h(B_1, B_2) = F(h(B_1), B_2)$



- Let $M' = (M, X)$
 - Then $h(K, M') = F(h(K, M), X)$
 - Attacker can compute HMAC of M' without K



- Is $h(M, K)$ better?
 - Yes, but... if $h(M') = h(M)$ then we might have
 $h(M, K) = F(h(M), K) = F(h(M'), K) = h(M', K)$

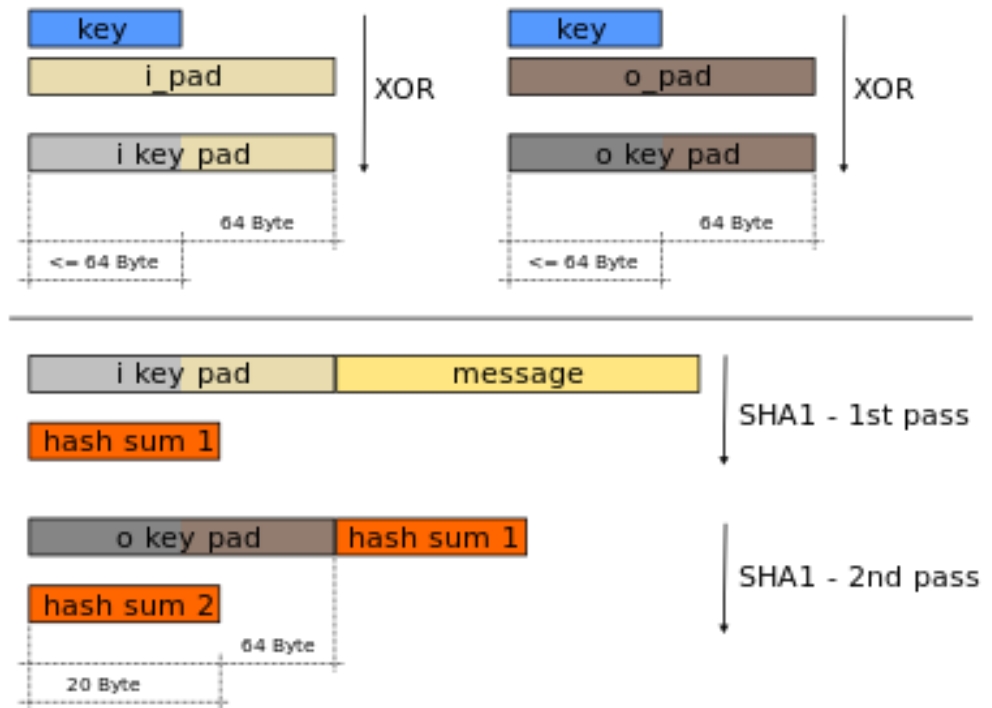
The Right Way to HMAC

- Described in RFC 2104
- Let B be the block length of hash, in bytes
 - B = 64 for MD5 and SHA-1 and Tiger
- $\text{ipad} = 0x36$ repeated B times
- $\text{opad} = 0x5C$ repeated B times

- Then

$\text{HMAC}(M, K) =$

$H(K \oplus \text{opad}, H(K \oplus \text{ipad}, M))$

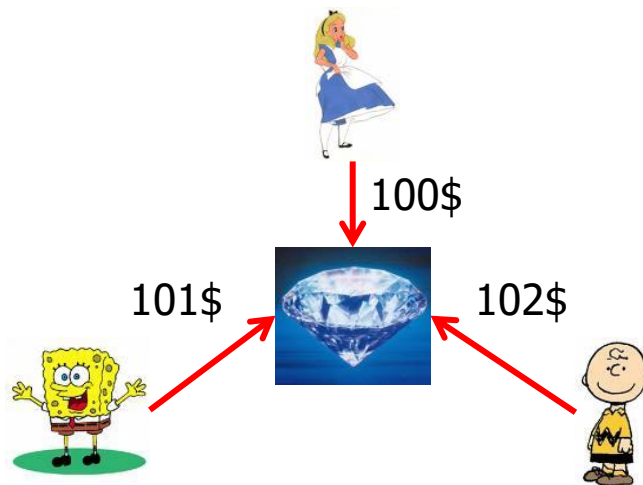


Hash Uses

- Authentication (HMAC)
- Message integrity (HMAC)
- Message fingerprint
- Data corruption detection
- Digital signature efficiency
- Blockchain (mining)

Online Auction

- Suppose Alice, Bob and Charlie are bidders
- Alice plans to bid A, Bob B and Charlie C
- They don't trust that bids will stay secret



Online Auction

- Solution
 - Alice, Bob, Charlie submit **hashes** $h(A)$, $h(B)$, $h(C)$
 - All hashes received and posted online
 - Then bids A , B and C revealed
- Hashes don't reveal bids (**one way**)
- Can't change bid after hash sent (**collision-resistant**)

